PRODUCT REQUIREMENTS DOCUMENT

Serverless Financial Data Pipeline

AWS Lambda • API Gateway • Kinesis • DynamoDB • S3 • React • GitHub Actions

Version	Author	Status	Date
v1.0	**Aman Srivastava**	**Draft**	**Spring 2026**

1. Project Overview

1.1 Purpose

The Serverless Financial Data Pipeline is a cloud-native web application that ingests live stock and cryptocurrency market data, processes it through an event-driven AWS backend, and surfaces actionable insights through an interactive React dashboard. The project is designed to demonstrate production-grade cloud engineering competencies: serverless compute, streaming data, infrastructure-as-code, and CI/CD automation.

1.2 Problem Statement

Financial data is high-frequency, latency-sensitive, and inherently time-series in nature. Most hobby projects that consume market data do so via batch polling, which is architecturally shallow and fails to demonstrate real cloud engineering skill. This project solves that by:

- Replacing polling with event-driven ingestion using AWS Kinesis Data Streams
- Using AWS Lambda for compute, eliminating the need for always-on servers
- Storing processed data in DynamoDB (hot path) and S3 (cold/archival path)
- Exposing data via API Gateway with a React frontend for real-time visualization
- Automating deployment entirely through GitHub Actions CI/CD pipelines

1.3 Goals & Non-Goals

In Scope

- Live ingestion of stock & crypto price data
- Serverless processing via AWS Lambda functions
- Kinesis Data Stream for real-time event ingestion
- DynamoDB for low-latency hot storage
- S3 for raw event archival
- REST API via API Gateway
- React + D3.js / Recharts frontend dashboard
- GitHub Actions CI/CD for Lambda deploys

Out of Scope

- Order execution or brokerage integration
- User authentication (v1)
- Mobile app
- ML-based price prediction (future phase)
- Multi-region deployment
- Paid data providers (use free APIs only)

- AWS CloudWatch metrics & alarms
- Free-tier-first architecture

2. System Architecture

2.1 High-Level Design

The pipeline follows an event-driven, serverless architecture with four logical layers:

Ingest	Stream	Process	Store	Serve	Visualize
Lambda Fetcher (cron trigger)	Kinesis Data Stream	Lambda Processor	DynamoDB + S3	API Gateway + Lambda	React + Recharts

2.2 AWS Services & Justification

Service	Role	Free Tier / Cost Notes
AWS Lambda	Compute for all pipeline stages	1M free requests/month; ~$0 for dev workloads
Kinesis Data Streams	Real-time event ingestion	Free tier: 1 shard, 1MB/s; ~$0.015/hr if exceeded
DynamoDB	Hot path time-series storage	25GB + 25 WCU/RCU free permanently
S3	Raw event archival (cold path)	5GB free; pennies per GB beyond
API Gateway	REST API for frontend	1M calls/month free for 12 months
CloudWatch	Logs, metrics, alarms	10 custom metrics free; logs ~$0.50/GB
EventBridge / CloudWatch Events	Cron trigger for fetcher Lambda	Free for scheduled rules
GitHub Actions	CI/CD pipeline	2,000 min/month free on public repos

2.3 Data Flow

Step-by-step data flow through the pipeline:

1. EventBridge fires a cron rule every 1 minute, triggering the Fetcher Lambda
2. Fetcher Lambda calls Yahoo Finance / CoinGecko REST APIs for configured tickers
3. Each price event is serialized to JSON and published to a Kinesis Data Stream shard
4. Kinesis triggers the Processor Lambda via event source mapping (batch size 10)
5. Processor Lambda writes normalized records to DynamoDB and raw events to S3
6. API Gateway routes GET /prices/{ticker} and GET /prices/{ticker}/history to a Query Lambda
7. Query Lambda reads from DynamoDB with GSI on ticker + timestamp for efficient range queries
8. React frontend polls the API every 30s and renders live charts with Recharts + D3.js

3. Feature Requirements

3.1 Feature Priority Matrix

ID	Feature	Priority	Effort	Resume Value
F-01	Kinesis ingestion pipeline	P0 — Must Have	Medium	★★★★★ Differentiator
F-02	Lambda Fetcher (cron-triggered)	P0 — Must Have	Low	★★★★ Serverless core
F-03	DynamoDB hot storage	P0 — Must Have	Low	★★★★ Standard cloud skill
F-04	S3 archival (raw events)	P0 — Must Have	Low	★★★ Storage tiering
F-05	API Gateway REST endpoints	P0 — Must Have	Low	★★★★ API design signal
F-06	React dashboard + Recharts	P0 — Must Have	Medium	★★★★ Full-stack proof
F-07	GitHub Actions CI/CD	P1 — Should Have	Low	★★★★★ Rarely seen on undergrad resumes
F-08	CloudWatch alarms & dashboard	P1 — Should Have	Low	★★★★ Observability signal
F-09	Multi-ticker support + filtering	P1 — Should Have	Medium	★★★ UX completeness
F-10	Infrastructure as Code (SAM/CDK)	P2 — Nice to Have	High	★★★★★ Senior-level signal
F-11	Price change alerts (SNS/SES)	P2 — Nice to Have	Medium	★★★ Event-driven bonus
F-12	JWT auth (Cognito)	P3 — Future	High	★★ Scope risk

3.2 Core Feature Specifications

F-01: Kinesis Ingestion Pipeline

- Create a Kinesis Data Stream with 1 shard (free tier)
- Partition key = ticker symbol (ensures ordering per asset)
- Retention period = 24 hours (default, free)
- Processor Lambda configured with EventSourceMapping, batch size 10, bisect-on-error enabled
- Dead letter queue (SQS) for failed batches to prevent data loss
- This is the most resume-valuable feature — call it out explicitly in your bullet points

F-02: Lambda Fetcher

- Runtime: Python 3.12

- Trigger: EventBridge cron rule — rate(1 minute)
- Data sources: Yahoo Finance (yfinance library) for stocks; CoinGecko public API for crypto
- Tickers configured via Lambda environment variable: e.g. AAPL,TSLA,BTC-USD,ETH-USD
- Each tick published as: { ticker, price, volume, timestamp, source }
- Error handling: catch API failures, log to CloudWatch, don't crash the invocation
- Cold start mitigation: keep dependencies minimal, use Lambda layers for yfinance

F-05: API Gateway Endpoints

Endpoint	Method	Description
/prices/{ticker}	GET	Latest price record for a given ticker
/prices/{ticker}/history	GET	Historical records; supports ?start=&end= query params
/tickers	GET	List of all configured tickers
/prices/compare	GET	Multi-ticker price comparison for charting

F-06: React Dashboard

- Stack: React + TypeScript + Tailwind CSS + Recharts
- Components: LivePriceTicker, PriceChart (line/candlestick), TickerSelector, AlertBanner
- Polling interval: 30 seconds via useEffect + setInterval
- Charts: real-time line chart for price, bar chart for volume, percentage change card
- Responsive layout — desktop-first, mobile-functional
- Hosted on S3 static site or Vercel (free tier)

4. Data Models

4.1 DynamoDB Schema

Table name: FinancialPriceEvents

Attribute	Type	Key Type	Notes
ticker	String	Partition Key	e.g. AAPL, BTC-USD
timestamp	String (ISO)	Sort Key	ISO 8601, enables range queries
price	Number	—	Current price in USD
volume	Number	—	24h trading volume
pct_change	Number	—	% change from previous record
source	String	—	yfinance or coingecko
ttl	Number (epoch)	—	DynamoDB TTL — expire after 7 days to control costs

4.2 S3 Object Structure

Raw events are stored in S3 using a Hive-style partitioned prefix for cheap Athena querying in future:

```
s3://financial-pipeline-raw/events/year=2026/month=05/day=12/hour=14/{uuid}.json
```

5. Technical Stack

Backend / Cloud

- Python 3.12 (Lambda runtime)
- AWS Lambda (fetcher, processor, query)
- AWS Kinesis Data Streams
- AWS DynamoDB
- AWS S3
- AWS API Gateway (REST)
- AWS CloudWatch
- AWS EventBridge (cron)
- AWS SQS (dead letter queue)
- boto3 SDK
- yfinance, requests libraries

Frontend / DevOps

- React 18 + TypeScript
- Tailwind CSS
- Recharts + D3.js
- Vite (build tool)
- GitHub Actions (CI/CD)
- AWS SAM or CDK (IaC — P2)
- pytest (Lambda unit tests)
- ESLint + Prettier
- Vercel or S3 static hosting

6. CI/CD Pipeline (GitHub Actions)

6.1 Pipeline Stages

The GitHub Actions workflow triggers on push to main and PRs. This is a key differentiator on your resume — most undergrad projects have zero CI/CD.

Stage	Action	Tools
Lint	Run flake8 on Lambda code, ESLint on frontend	flake8, ESLint
Test	Run pytest unit tests for Lambda handlers	pytest, moto (AWS mock)
Build	Package Lambda zips, build React bundle	pip, Vite
Deploy Lambdas	aws lambda update-function-code for each handler	AWS CLI, GitHub Secrets
Deploy Frontend	Upload dist/ to S3 static bucket	AWS CLI
Smoke Test	curl API Gateway endpoint, assert 200	curl, bash

⚠ Cost Safety: Set a CloudWatch billing alarm at $5 before building anything. Add `AWS_REGION=us-east-1` and never commit AWS credentials to GitHub — use GitHub Secrets for AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY.

7. Resume Bullet Points

7.1 Suggested Resume Entry

Once built, use these four bullets — adjust metrics to match your actual measurements:

Serverless Financial Data Pipeline

- Architected an event-driven AWS pipeline ingesting live stock and crypto prices via Kinesis Data Streams, processed by Lambda functions and stored across DynamoDB (hot path) and S3 (archival)
- Deployed three Lambda functions (fetcher, processor, query) triggered by EventBridge cron rules and Kinesis event source mappings, achieving <200ms end-to-end ingestion latency
- Built a React + TypeScript dashboard with Recharts visualizing real-time price feeds and historical trends for 10+ configurable tickers via REST API Gateway endpoints
- Automated deployment with a GitHub Actions CI/CD pipeline covering lint, pytest unit tests (moto-mocked AWS), Lambda packaging, and S3 frontend publishing on every push to main

8. Development Milestones

Week	Milestone	Deliverables
Week 1	AWS Setup & Fetcher	AWS account, billing alarm, IAM roles, Fetcher Lambda + EventBridge, data publishing to Kinesis
Week 2	Processor + Storage	Processor Lambda with Kinesis trigger, DynamoDB schema + writes, S3 archival, CloudWatch logs
Week 3	API Layer	API Gateway REST endpoints, Query Lambda, DynamoDB GSI for range queries, basic curl testing
Week 4	React Frontend	React + Tailwind scaffold, Recharts live chart, polling logic, multi-ticker support
Week 5	CI/CD + Polish	GitHub Actions pipeline, pytest + moto tests, CloudWatch alarms, README with architecture diagram

9. Risks & Mitigations

Risk	Severity	Mitigation
Unexpected AWS charges from runaway Lambda/Kinesis	High	Set $5 billing alarm on Day 1; tear down Kinesis shard when not actively developing

yfinance rate limiting or API deprecation	Medium	Cache last response; fall back to CoinGecko; add error handling so Lambda doesn't fail loudly
DynamoDB costs exceeding free tier	Low	Enable TTL (7-day expiry) on all records; monitor item count weekly
Kinesis shard hours exceeding free tier	Medium	Delete stream when not in use; use on-demand mode in production
GitHub Actions secrets exposure	High	Never hardcode keys; use GitHub Secrets; restrict IAM role to least-privilege